Where Should a Robot Swarm See Its Neighbors? Communication-Range Transfer in a GNN Coverage Policy

Loïc Poisson **Harold Gardon** **Claire Fitussi**
Stanford University
{lpoisson, gardon21, cfitussi}@stanford.edu

Status in one paragraph. Since the proposal we have (i) finished the diagnosis of LPAC's failure when the communication range grows, (ii) implemented and trained six variants of the policy (three seeds each, 25 training runs on a FarmShare L40S GPU), and (iii) found the first trained model whose coverage improves when robots hear farther: a GNN layer that reads *bounded* relative geometry on the edges (V4), and (iv) answered with new measurements the three questions raised on our earlier draft: dropping versus clamping the far robots, directed links in mixed-range teams, and why GATv2 underfits (Sec. 5). We also learned two methodological lessons that change how we will report results. This draft is organized as the final blog post will be: motivation, data, model, results, and what remains.

1 Problem and motivation

In *coverage control*, N robots spread over an area so that their sensors watch the regions that matter, given by an importance density field (IDF) ϕ unknown in advance [Cortés et al., 2004]. Each robot sees only a 64×64 m footprint and talks only to robots within its *communication range* R. The communication network is the graph $G_R = (V, E_R)$, with an edge (i, j) whenever $\|p_i - p_j\| \leq R$, so decentralized control is a graph learning problem: each robot decides what to send to its neighbors and how to combine what it receives [Tolstaya et al., 2020].

LPAC [Agarwal et al., 2025] is a recent GNN policy for this task. Its authors report that it degrades when R grows (their Sec. VI-H, Fig. 20) and hypothesize that aggregating many messages loses information. This matters in practice: a radio's range changes with terrain and hardware, and a fleet may mix robots with different radios. A policy that gets *worse* when robots communicate *more* cannot be deployed safely. Our question is therefore: *where should neighbor geometry enter a GNN policy, and at which scale, so that the policy transfers across communication ranges and benefits from longer ones?*

2 Data, processing and metric

Dataset. We use the public **CoverageControl** library [KumarRobotics, 2024] (GPL-3.0, release v1.3.1): a simulator of 32 robots in a 1024×1024 m world whose IDFs are mixtures of 32 random Gaussians, the published LPAC checkpoint, its 11 evaluation environments, and scripts that generate imitation data from a *clairvoyant* centroidal-Voronoi (CVT) controller that knows ϕ and every position.

Processing. One sample is the state of all 32 robots at one time step: four 32×32 maps per robot (local importance, two neighbor-map channels, obstacles), the positions, the graph G_R and the clairvoyant velocity as target. As in the paper, a training set has 100,000 samples. We generated sets at 128, 256, 512 and 1024 m, merged them into a *mixed-range* set, and wrote a redrawing tool

CS224W: Machine Learning with Graphs, Stanford University, Autumn 2026. Project milestone (type 1: real-world application).

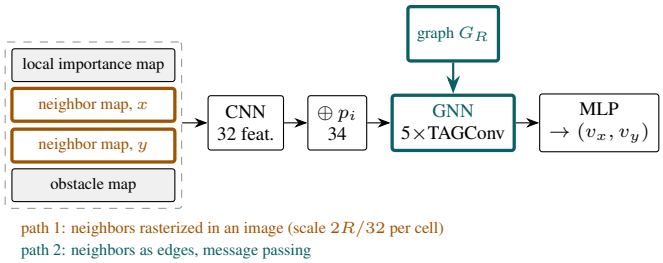


Figure 1: LPAC on one robot. Neighbors enter twice: drawn in a 32×32 neighbor map read by the CNN (path 1), and as edges of G_R read by the GNN (path 2).

that rebuilds the neighbor maps of an existing set at any scale (it recovers each sample's range from its stored edge weights), so a new map encoding does not require re-simulating. We also generated 40 new evaluation environments.

Task and metric. The policy maps local observations and messages to a velocity (node-level regression on G_R), trained by imitation with a mean-squared error. We evaluate it in closed loop for 600 steps with the paper's metric: the coverage cost $J = \int \phi(q) \min_i \|q - p_i\|^2 \, dq$ averaged over the episode and divided by the clairvoyant's cost on the same environment (lower is better). We vary the *test-time* range $R \in \{128, 256, 512, 768, 1024\}$ m. Baselines are the centralized CVT (about 2.3–2.5) and the published model.

Protocol and noise. Evaluation is deterministic on Intel CPU nodes (AMD nodes round convolutions differently). Differences between models are paired by environment. We report the 40 new environments as the primary set: two runs of the same recipe differ there by 0.01–0.05, against 0.08–0.15 on the 11 released ones (Sec. 4).

3 Model design

LPAC. Each robot runs the same network (Fig. 1). A CNN encodes the four maps into 32 features, the position is appended, five TAGConv layers [Du et al., 2017] with $K = 3$ exchange information over G_R, and an MLP outputs the velocity:

$$X^{(\ell+1)} = \mathrm{ReLU}\Big(\textstyle\sum_{k=0}^{K} \big(D^{-1/2} A D^{-1/2}\big)^k X^{(\ell)} W_k^{(\ell)} \Big). \tag{1}$$

In the *neighbor map*, robot j within range is written into cell $c_{ij} = \mathrm{round}\big((p_j - p_i)\frac{32}{2R} + 15.5\big)$ with value $(p_j - p_i)/R$. The map therefore zooms out with R: a cell is 8 m wide at 128 m and 64 m at 1024 m, and a model trained at one range sees images at test time that it never saw in training.

Variants. All variants keep the TAGConv stack and the training recipe (imitation, Adam, best validation checkpoint) and change only how neighbor geometry is encoded. *Mixed* means trained on the merged 128–1024 m set.

- **A**: LPAC with the neighbor map drawn at a fixed 256 m scale (cells of 16 m, neighbors within $\min(R, 256)$ m).
- **V1**: no neighbor map. **V2**: GATv2 [Brody et al., 2022] with $p_j - p_i$ as edge feature instead of TAGConv.
- **V3**: V1 plus a residual PointNet layer [Qi et al., 2017] before the TAGConv stack, with edge input $(p_j - p_i)/256$ m.
- **V4** (bounded edges): V3 with an edge input that stays in $[-1, 1]$ at every range. With $d_{ij} = \|p_j - p_i\|$,

$$e_{ij} = \Big[\tfrac{p_j - p_i}{d_{ij}},\ \tanh\big(\tfrac{d_{ij}}{256}\big) \Big], \qquad h_i = x_i + \mathrm{ReLU}\Big(g\big(\max_{j \in \mathcal{N}(i) \cup \{i\}} f([x_j, e_{ij}]) \big) \Big), \tag{2}$$

with f, g two-layer MLPs, max aggregation (invariant to the number of neighbors, about 2 at 128 m and 30 at 1024 m) and $e_{ii} = 0$ for the self-loop.

Table 1: Cost / clairvoyant cost (lower is better), 40 new environments, mean over three training seeds (six runs for V5). The inference-time fix is shown for reference; all other rows are trained models used as is.

Model	128 m	256 m	512 m	768 m	1024 m
LPAC published (trained 128 m)	1.96	1.70	2.87	–	6.15
+ fixed 256 m map at inference	–	1.70	1.71	–	1.77
Centralized CVT	2.33	2.33	2.33	–	2.33
A: fixed map, mixed	2.09	1.91	1.91	1.91	1.92
V3: PointNet edges, trained 128 m	2.13	2.03	2.25	2.39	2.44
V4: bounded edges, trained 128 m	2.05	1.87	1.90	1.90	1.90
V4: bounded edges, mixed	2.14	1.87	**1.81**	**1.81**	**1.82**
V5: multi-scale maps, mixed	2.09	1.89	1.89	1.90	1.90
V6: V5 + V4, mixed	2.12	1.91	1.87	1.88	1.89

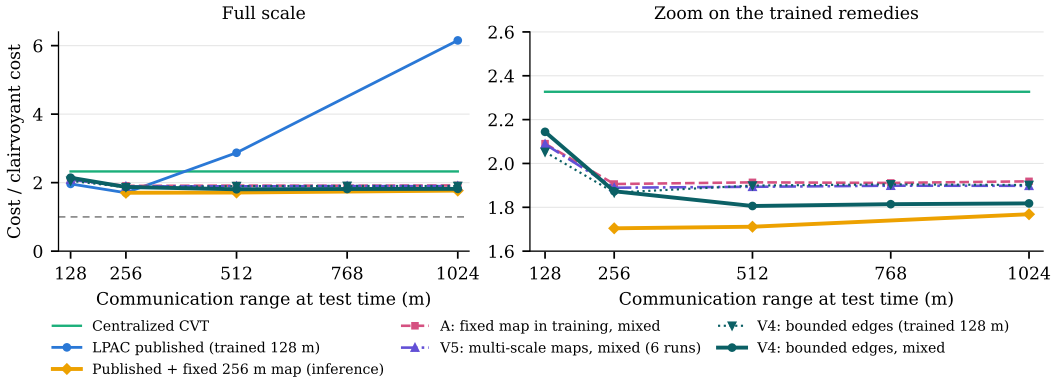


Figure 2: Cost against test-time range on the 40 new environments. Left: the published model collapses. Right: every trained remedy removes the collapse; only V4 trained on mixed ranges gets better beyond 256 m.

- **V5** (multi-scale maps): LPAC with three neighbor-map pairs at fixed scales $s \in \{256, 512, 1024\}$ m, each holding the neighbors within $\min(R, s)$, so the CNN reads 8 channels.
- **V6**: V5 plus the V4 edge layer.

4 Results so far

1. The collapse comes from the image, not from the graph. Retrained, LPAC matches the published model (1.74 vs 1.69 at 256 m). Trained at 128 m, it reaches 5.22 at 1024 m (6.15 on the 40 new environments), worse than the centralized CVT. A factorial intervention at inference separates the two paths. Capping G_R to the 4 nearest neighbors leaves the cost at 5.11. Redrawing only the neighbor map at a fixed 256 m scale, with the full graph, brings it to 1.71 (paired $+0.02 \pm 0.16$ against 256 m). This holds for every model and on the 40 new environments, and the effective rank of TAGConv outputs never collapses, which rules out over-smoothing. The paper's hypothesis, a lossy aggregation of many messages, is therefore not the cause in transfer.

2. Bounded geometry on the edges removes the collapse. Table 1 and Fig. 2 compare the trained remedies. Removing the map (V1) costs 0.75 at 256 m, and GATv2 on raw relative positions (V2) underfits, a claim we test in Sec. 5. V3's PointNet layer degrades with range because its inputs grow with R. V4 replaces them with a unit direction and a saturating distance: *trained at 128 m only*, it no longer collapses (1.90 at 1024 m, three seeds) and beats V3 at every range, beyond seed noise.

3. A first model that benefits from a longer range. Trained on mixed ranges, V4 is the only model whose cost *decreases* from 256 to 1024 m: -0.055 (SE 0.016, 27/40 environments better) on

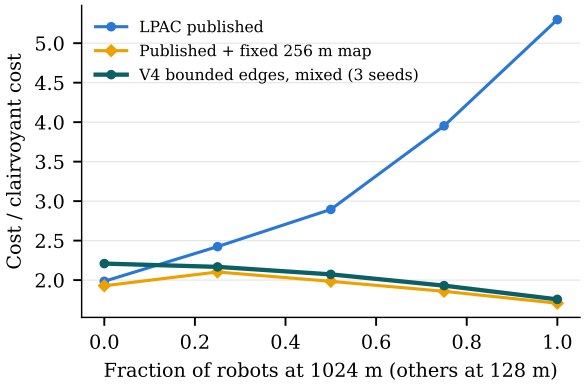


Figure 3: Heterogeneous teams on the 11 released environments: a fraction of the robots hears at 1024 m, the others at 128 m (8 random draws for LPAC, 4 per seed for V4), with the reception rule of Sec. efsec:ta, Q2.

the new environments and -0.12 (SE 0.03, 10/11) on the released ones, in each of the three seeds. A and V5 are flat, which means they tolerate a longer range without using it. V6 behaves exactly like V5 (within 0.03 at every range), so the two encodings do not add up. No trained model yet reaches LPAC's 256 m level: at 256 m all stay 0.05–0.09 behind LPAC retrained at that range (1.82 on the new environments).

4. Heterogeneous teams. When a fraction f of the robots hears at 1024 m and the rest at 128 m (Fig. 3), the published model's cost grows from 1.98 to 5.30. With V4 (mixed), each robot that hears farther helps: the cost falls at every step of f for each seed, by 0.45 in total (SE 0.08, 11/11 environments). This matches the inference-time fix within noise ($+0.05$ to $+0.09$, SE 0.08) for $f \geq \frac{1}{4}$, without any test-time change. V4 remains 0.28 behind when every robot is at 128 m, its weakest range.

5. Two methodological lessons. *(a) Run-to-run noise.* Training is not bit-reproducible at a fixed seed (GPU scatter operations, parallel data loading). Two launches of V5 with the same seed differ by 0.08–0.15 at 256 m on the 11 released environments, more than the spread between seeds of one launch. A first set of three V5 runs appeared to match LPAC at 256 m (1.78); pooled with three more, it is 0.10 behind. We now report the 40 new environments as primary and will use six or more runs per recipe for any claim at the 0.05 level. *(b) Validation loss does not predict coverage.* Map-based models reach their best validation MSE at epoch 4–6, after which it rises from 0.46 to 0.54 while the coverage cost does not move: epoch-25 checkpoints of V5 and V6 match their best-validation checkpoints within 0.04. Conversely, the repaired GATv2 of Sec. 5 has the lowest validation MSE of all our runs (0.488, against 0.536 for V1) but no better coverage than V1. Imitation loss is therefore a poor model-selection signal for this task.

5 Backing up three claims

Q1. Are far neighbors dropped or clamped? In the fixed-scale map (A, and the inference-time fix of Table 1), robots beyond 256 m are *dropped*. We added the alternative: project each of them onto the 256 m circle along its direction, into a border cell, with its value clipped to norm 1 (*clamp*). Within 256 m the two maps are identical (unit-tested). On the published model and the 40 new environments, clamping is worse at every range (Table 2, top): $+0.37$, $+0.70$, $+0.88$ at 512, 768, 1024 m (paired SE 0.06–0.08; clamp better in at most 5/40 environments), although both beat the published map by far. The reason is the input's magnitude: at 1024 m, clamping piles up to 30 robots onto one ring of border cells, giving a largest cell value of 4.29 against 1.10 when dropping, four times outside the training distribution. Dropping itself costs little: $+0.06$ at 1024 m against the same model at its 256 m optimum (SE 0.035). This sharpens result 1: what breaks the policy at long range is the *distortion* of its input, not the loss of the far robots.

Table 2: Answers to Q1 and Q2 (cost / clairvoyant cost, lower is better). Top: published model, 40 new environments, three ways to draw the neighbor map at inference. Bottom: V4 (mixed), teams of 128 m and 1024 m robots with a fraction f at 1024 m, 11 released environments, 3 seeds $\times$ 4 random draws; rule rows are paired differences against reception (SE 0.02–0.04).

Map at inference	512 m	768 m	1024 m
Drop far robots	**1.71**	**1.73**	**1.77**
Clamp to border	2.08	2.43	2.65
As published	2.87	–	6.15

Link rule	$f=0$	$\frac{1}{4}$	$\frac{1}{2}$	$\frac{3}{4}$	1
Reception ($d < r_i$)	2.21	2.17	2.07	1.93	1.76
Transmission ($d < r_j$)	=	$\mathbf{-0.17}$	$\mathbf{-0.23}$	$\mathbf{-0.12}$	=
Either reaches (OR)	=	-0.22	-0.26	-0.17	=
Both reach (AND)	=	-0.01	-0.05	-0.01	=

Q2. Directed links in mixed-range teams. Our heterogeneous experiments (result 4) used *reception*: robot i hears j when $d_{ij} < r_i$. Under a transmit-power radio, the natural rule is *transmission*, the directed graph in which a 1024 m robot reaches a 128 m one: $j \to i$ when $d_{ij} < r_j$. We also tested the symmetric AND and OR rules. Only the graph changes; the four rules coincide exactly at $f = 0$ and $f = 1$, as they must. For V4, transmission gains 0.12–0.23 over reception, in 11/11 environments at every f (Table 2, bottom). OR gains slightly more, but it lets a 128 m robot reach a robot 1024 m away, so we treat it as an optimistic bound. AND removes exactly the edges that transmission adds and changes nothing. The gain therefore comes from short-range robots *listening* to long-range ones. At $f = \frac{3}{4}$ the team is within 0.05 of an all-1024 m team. The published model, in contrast, *loses* 0.4–0.8 under transmission with its own map, and is neutral with the fixed-scale map. Our hypothesis, not yet measured directly, is that the added edges bring in robots that its neighbor map does not show.

Q3. Is GATv2 underfitting, untuned, or structurally unfit? V2's training loss is stuck: it moves by 0.02 between epochs 10 and 30, while V1's keeps falling (Table 3). Five single-seed runs on the 128 m set separate the causes. A $10\times$ learning rate changes nothing. Pre-norm residual blocks with LayerNorm and V4's bounded edge features (V2res) take the loss from 0.676 to 0.432, closing 86% of the gap to V1. Given that repair, 15 blocks train no better than 5. A TAGConv control with the same 5-hop receptive field as V2 (V1, $K{=}1$) matches V1 exactly, so reach was never the issue. Adding the node degree, which softmax attention discards, does not help. Our answer is *underfitting caused by the conditioning of the original configuration, not by attention as such*. The repair also shows up in coverage: -0.70 against V2 at 256 m (SE 0.19, 10/11 environments), bringing GATv2 level with V1. It does not make attention competitive with the map (2.26 vs. 1.69) and it does not stop the degradation with range: $+0.85$ from 256 to 1024 m (SE 0.16, worse in 10/11 environments). V4 reads the same bounded edge features but does not degrade, so the difference is likely the aggregation. V4 takes a maximum over neighbors, which does not depend on how many there are; softmax attention takes a weighted mean, which is diluted as the neighborhood grows from about 2 to about 30 robots. Caveats: one seed, a remaining 0.04 gap in training loss to V1, and V2res changes three things at once. A run that adds only the bounded edges (V2B) is queued to isolate the cause.

6 Code

The repository[1] contains: `lpac_study/`, a Python package around CoverageControl and PyTorch Geometric [Fey and Lenssen, 2019] with the closed-loop evaluation (`episode.py`, `sweep.py`), the inference-time interventions on the graph and the map (`factorial.py`, `neighbourhood.py`), the variants V0–V6 (`variants.py`), heterogeneous teams with four link rules (`heterogeneous.py`), the specialist mixture (`mixture.py`) and the analyses (effective rank, CKA, checkpoint selection);

[1] https://github.com/loicpoisson/lpac-communication-range (private until the final submission; access on request).

Table 3: Q3: diagnosing GATv2 (V2). Training MSE at epochs 10/20/30 (128 m set, one seed, 30 epochs) and coverage cost on the 11 released environments (V1 and V2: mean of three seeds).

Run	Change from V2	Train MSE, ep. 10 / 20 / 30	256 m	1024 m
V2	–	0.699 / 0.685 / 0.676	2.96	3.31
V2lr	learning rate $\times 10$	0.698 / 0.686 / 0.679		
V2res	residual, LayerNorm, V4 edges	0.522 / 0.467 / **0.432**	**2.26**	3.11
V2deep	15 V2res blocks	0.533 / 0.467 / 0.433		
V2res + degree	$\log(1 + \deg)$ as node feature	0.533 / 0.476 / 0.447		
V1	TAGConv $K{=}3$, no attention	0.540 / 0.462 / 0.392	2.48	3.14
V1, $K{=}1$	5 hops, like V2	0.530 / 0.455 / 0.392		

`training/`, data generation, map redrawing and the Slurm launchers for FarmShare; `tests/`, unit tests of the variants, the clamped map, the link rules and the heterogeneous training set; the result tables and a running journal of every experiment. An early notebook (`lpac_eval.ipynb`) runs the evaluation of the published model; it will grow into the Colab.

7 Plan to the final submission

1. **Firm up the main claims** (queued): six runs of V4 mixed and three of LPAC retrained at 256 m, so that the 256 m gap and the range gain are measured against run-to-run noise.
2. **Train for mixed-range teams** (queued): V4 trained on heterogeneous teams wired by the transmission rule of Q2, against V4 mixed rewired only at test time.
3. **Close V4's gap at short range**: V4 is weak at 128 m (2.14 vs. 2.05 when trained at 128 m only). The fixed-map LPAC, which has no edge features, loses a similar amount at 128 m ($+0.06$, SE 0.03, 40 environments) when trained on mixed ranges instead of 128 m, so we attribute the gap to the small share of 128 m samples (one quarter) rather than to V4's edge features. A run trained on a set with half its samples at 128 m is queued to test this.
4. **Isolate the GATv2 repair** (queued): V2B, which changes only the edge features. Then test the aggregation hypothesis of Q3: V2res with max instead of softmax aggregation should stop degrading with range if aggregation is the cause.
5. **Information loss vs. distribution shift**: draw the same neighbors in a finer map to test whether the 256 m map discards useful information.
6. **Deliverables**: the Colab (data loading, model definitions in PyG, training on a reduced set, closed-loop evaluation of the released checkpoints) and the Medium post built on Figs. 1–3.

References

Saurav Agarwal, Ramya Muthukrishnan, Walker Gosrich, Vijay Kumar, and Alejandro Ribeiro. LPAC: Learnable perception-action-communication loops with applications to coverage control. *IEEE Transactions on Robotics*, 41, 2025. arXiv:2401.04855.

Shaked Brody, Uri Alon, and Eran Yahav. How attentive are graph attention networks? In *International Conference on Learning Representations*, 2022.

Jorge Cortés, Sonia Martínez, Timur Karataş, and Francesco Bullo. Coverage control for mobile sensing networks. *IEEE Transactions on Robotics and Automation*, 20(2):243–255, 2004.

Jian Du, Shanghang Zhang, Guanhang Wu, José M. F. Moura, and Soummya Kar. Topology adaptive graph convolutional networks. *arXiv preprint arXiv:1710.10370*, 2017.

Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

KumarRobotics. CoverageControl: A library for coverage control with robot swarms, release v1.3.1. https://github.com/KumarRobotics/CoverageControl, 2024.

Charles R. Qi, Li Yi, Hao Su, and Leonidas J. Guibas. PointNet++: Deep hierarchical feature learning on point sets in a metric space. In *Advances in Neural Information Processing Systems*, 2017.

Ekaterina Tolstaya, Fernando Gama, James Paulos, George Pappas, Vijay Kumar, and Alejandro Ribeiro. Learning decentralized controllers for robot swarms with graph neural networks. In *Conference on Robot Learning*, volume 100 of *Proceedings of Machine Learning Research*, pages 671–682, 2020.